Knowledge Distillation and Deployment of a Command Parser on a Local Cloud

CS3081 Cloud Computing — Mini Project Report

Manas Trivedi (230101073)

Repository: https://github.com/Manas-Trivedi/cc-kd-command-parser (assessed commit: tag `final`)

Overview. A teacher LLM (phi3.5, 3.8 B, on Ollama) labels 3,000 voice-assistant commands with a four-key JSON frame. A 70 M-parameter student (Pythia-70M) is fine-tuned on those replies (sequence-level distillation) and served by a FastAPI service on a k3d cluster. Weights live in SeaweedFS object storage and are downloaded by every pod at start-up. All evidence is in `results/`. **Notation:** values marked $^{\mathbf{M}}$ are measured on my laptop (Apple M3 Pro, 11 cores, 18 GB; Docker via OrbStack with 9 GB); values marked $^{\mathbf{A}}$ are assumptions or given prices.

```
laptop: Ollama phi3.5 (Apple GPU)  <-- HTTP --  ns pipeline: Job teacher-label (Indexed, 3 pods, SHARD_INDEX = index)
                                                  | raw replies -> s3://datasets/teacher/shard-{0,1,2}.jsonl
ns data: SeaweedFS (S3 API, PVC)  datasets/{commands_pool.jsonl, teacher/, v1/train.jsonl}   models/{v1/, v2/}
                                                  | boto3 download at pod start (keys from Secret s3-credentials)
localhost:8080 -> Traefik Ingress /parse -> Service app/parser -> Deployment parser: 2 pods (HPA 1-2 @ 70% CPU)
```

1 Task 1 — Object storage and parallel teacher labelling

Setup. `cluster/k3d.yaml` defines one server and one agent node, maps the Traefik load balancer to `localhost:8080` and the SeaweedFS S3 NodePort to `localhost:8333`. SeaweedFS runs as one pod (`weed server -s3`) on a PersistentVolumeClaim in namespace `data`. Its S3 identity file and the client keys are generated by `scripts/secrets.sh` into `.secrets/` (git-ignored) and loaded as Kubernetes Secrets, so no credential is in the repository. Buckets `datasets` and `models` exist and `datasets/commands_pool.jsonl` is uploaded (seed Job, or `scripts/s3.py`).

Teacher. Ollama runs natively on the laptop so phi3.5 uses the Apple GPU; pods reach it at `host.docker.internal:11434`. Options: `temperature 0, seed 42, num_predict 96`, and Ollama's `format: "json"`. The prompt (`prompts/teacher_prompt.txt`) lists the eight intents, the three slot rules and ten examples I wrote myself (none from the validation set). In a pilot on the first 80 pool commands, phi3.5 without JSON mode appended explanatory "Note:" paragraphs after the JSON in 13/80 replies$^{\mathbf{M}}$; with JSON mode 0/80 were invalid$^{\mathbf{M}}$, so I kept JSON mode. Replies are stored exactly as generated.

Sharding (worker/worker.py). Each worker reads `SHARD_INDEX` and `SHARD_COUNT`, keeps the pool lines whose 0-based line number i satisfies $i \bmod \text{SHARD_COUNT} = \text{SHARD_INDEX}$, and writes `{id, text, teacher_raw}` lines to its own object `datasets/teacher/shard-<k>.jsonl`. The workers run as one Kubernetes *Indexed Job* (`completions = parallelism = 3`) in namespace `pipeline`; Kubernetes gives each pod a distinct completion index 0..2, which the manifest passes in as `SHARD_INDEX`. Modulo (round-robin) sharding gives equal shard sizes (1,000 each) and mixes intents evenly, and because each line maps to exactly one remainder there can be no gap or overlap. Each worker re-uploads its shard every 20 commands and, on start, skips ids already in its shard object. This mattered: the Docker VM froze mid-run and had to be restarted; the Job re-created the three pods, and each resumed at 300/1,000 instead of starting over (`results/task1/label_resume_after_restart.txt`). Total wall time 76 min$^{\mathbf{M}}$ (3 parallel requests to one Ollama, ≈0.55 commands/s per worker$^{\mathbf{M}}$). `results/task1/shard_coverage.txt` confirms all 3,000 ids appear exactly once$^{\mathbf{M}}$.

Cleaning (worker/clean.py). Records are processed in id order; the first failed check is the reason. Survivors are de-duplicated on normalised text (lower case, single spaces), keeping the smallest id. Output is serialised deterministically (sorted keys, fixed separators), so re-running gives the same SHA-256; I re-ran it on the downloaded shards and got the same checksum$^{\mathbf{M}}$.

	Input	(a) not JSON	(b) wrong keys	(c) bad intent	(d) slot not in command	(e) duplicates	Kept
Records$^{\mathbf{M}}$	3,000	2	2	13	162	49	2,772

$2 + 2 + 13 + 162 + 49 + 2772 = 3000$. `train.sha256` = `a58e34e5f831aea4...c9ec5e` (full value in `data/v1/train.sha256`).

Most rejections are (d): the teacher paraphrases ("in two days" for "after two days") or invents (`"time": "now"`), which the copy-exactly rule forbids. 50 commands in the pool are repeats; one repeat was already rejected by (d), so (e) removes 49.

2 Task 2 — Training the student (sequence-level distillation)

Method. Each example is `Command: <text>\nJSON: <teacher JSON><eos>`; the loss is masked to the reply tokens, so the student learns p(teacher reply | command). Training uses a plain PyTorch loop (AdamW, linear warm-up/decay, gradient clip 1.0) on the CPU, on the first 1,500 cleaned records. `training/config.yaml` holds seed 42, learning rate 5×10^{-4}, batch 16, 10 epochs, max length 96 and the library versions (torch 2.5.1, transformers 4.46.3, ...); `make train` reads only this file and `make train SUBSET=50` trains on 50 records. Seeds (Python, NumPy, torch, data order) are set in code and deterministic algorithms are on: re-training v1 reproduced byte-identical predictions[M]. One epoch takes ≈31 s[M] on 11 CPU threads. Inference is greedy and stops at EOS or the first newline.

Versions. My first run used 8 epochs; it left one of 100 validation outputs as truncated JSON (99% validity[M], kept in `results/experiments/`). Because each invalid output becomes an HTTP 500 in the load test, I increased to 10 epochs: this is **v1** (seed 42). **v2** is identical except seed 7. Training loss (v1, `training/loss.csv`[M]): 0.808, 0.392, 0.219, 0.157, 0.107, 0.084, 0.049, 0.027, 0.012, 0.004. Weights are saved as float16 (141 MB instead of 282 MB; my upload link managed only ≈40 KB/s[M], and the first 226 MB Release upload timed out) and loaded back as float32 for inference; validation predictions are byte-identical to the float32 weights[M]. Running training twice from scratch gave the same weights file SHA-256 and the same losses[M] (`results/retrain_reproducibility.txt`). Model and tokenizer are in `s3://models/v1/` and `s3://models/v2/`. Each version is packed into a reproducible tarball (sorted entries, zero mtimes); `models/v1/model.sha256` is the SHA-256 of that tarball, which is the GitHub Release asset `model-v1.tar.gz` (`18a4f792...8731571a`); CI re-checks it with `sha256sum -c`.

Validation (100 commands)[M]	JSON valid	Intent	Date	Time	Place	Full frame
Teacher phi3.5 (same prompt/settings)	99	90	77	84	91	**54**
Untrained Pythia-70M	0	0	0	0	0	0
Student v1 (10 epochs, seed 42)	**100**	86	78	85	82	**45**
Student v2 (10 epochs, seed 7)	100	86	80	81	85	48
First attempt (8 epochs, seed 42)	99	83	72	83	81	40

What changed. The untrained model just continues text (`- name: "calendar", {`) and never produces a valid frame. After distillation the student always emits the schema and reaches 83% of the teacher's full-frame accuracy (45 vs 54) with 1/54 of the parameters; it agrees exactly with the teacher on 52/100 commands[M]. It loses most on intent (86 vs 90) and place (82 vs 91): place names are rare tokens it has seen few times. The student is *more* valid than the teacher: the teacher's one invalid output was `"intent": null`, a mistake the student, which only ever saw cleaned replies, never learned. Raw outputs and score reports are in `results/` (`teacher_validation.json`, `untrained_validation.json`, `student_v{1,2}_validation.json` and the matching `*_predictions.jsonl`).

Sequence-level vs logit-based distillation. *Sequence-level* KD (Kim & Rush, 2016) trains the student with ordinary cross-entropy on the *text the teacher generated* (here the teacher's greedy JSON reply): a single hard target sequence per input. *Logit-based* (word-level) KD trains the student to match the teacher's full *probability distribution over the vocabulary at every position* (e.g. KL divergence between softened teacher and student softmax outputs, as in Hinton et al. and TRL's GKD/DistillationTrainer), so it also learns how likely the alternatives were. *It is not possible here* because (i) the teacher is reached only through Ollama's text API, which returns generated text, not per-token logits over the vocabulary; and (ii) phi3.5 and Pythia use different tokenizers (32k vs 50k vocabularies), so their distributions are over different token sets and cannot be compared position by position without an alignment scheme. *Limitation of my approach:* the student only sees one hard answer per command, including the teacher's mistakes: the teacher scores 54% full-frame, so its errors (e.g. labelling "remind me" as `alarm_set`, or extra date words) are copied as if correct. The student's ceiling is the teacher, and a hard label carries no information about how uncertain the teacher was. Cleaning removes only *format* errors; wrong-but-valid labels stay in the training set.

Suitability of Pythia-70M. Apache-2.0 licensed and small enough to train in about 5 minutes on a laptop CPU, which makes it good for this demonstration. But it is a base model pre-trained on the Pile with no instruction tuning, and the model card states it is not intended for deployment; with 70 M parameters it memorises the training distribution (loss → 0.004) and generalises poorly to unseen place names. It is adequate for a fixed-format task, not for open-ended use.

3 Task 3 — Inference service in a container

`service/app.py` (FastAPI, one uvicorn worker) exposes `POST /parse`, `GET /healthz` (always 200 while the process runs) and `GET /readyz` (503 until the model is loaded, then 200). Configuration comes from `MODEL_URI`, `MODEL_VERSION`, `S3_ENDPOINT`, `MAX_NEW_TOKENS` and `FAULT_INJECT`; the S3 keys come from the Secret `s3-credentials`

via `envFrom`. At start-up a background thread lists `MODEL_URI`, downloads every object with boto3, loads the model and runs one warm-up generation. Every output is parsed and validated with `jsonschema` against `schema.json` (exact keys, intent enum, string or null slots); if invalid, the response is HTTP 500 `{"error": "invalid_model_output"}`. With `FAULT_INJECT=1` the output is replaced with `not json` before validation. Successful responses carry `X-Model-Version` and `X-Served-By` (pod name from the Downward API). Inference is serialised with a lock and uses one torch thread to match the 1-CPU limit. Nine unit tests cover the endpoints, headers, six kinds of invalid output and fault injection.

Image. `python:3.12.8-slim` + the CPU-only torch wheel from `download.pytorch.org/whl/cpu` (the default wheel pulls CUDA libraries) + a fully pinned `service/requirements.txt` (46 packages incl. torch, generated with `uv pip compile`). No weights are copied in; a `find` for `*.safetensors/*.bin` in the image returns $0^{\mathbf{M}}$. Size **1.23 GB**$^{\mathbf{M}}$. Build: `docker build -f service/Dockerfile .` (or `make images`).

Running under limits (`scripts/container_test.sh`, `-cpus 1 -memory 1g`; output in `results/task3/container_test.txt`). 50 sequential validation commands: 50/50 HTTP 200, median latency 408 ms, max 468 ms$^{\mathbf{M}}$. **Peak memory: 788.7 MiB** process peak RSS (`VmHWM`)$^{\mathbf{M}}$; the cgroup's `memory.peak` was 765.5 MiB and the highest `docker stats` sample 593.1 MiB$^{\mathbf{M}}$, all under the 1,024 MiB limit. The peak happens while the float16 file is converted to float32 at load; steady state is ≈590 MiB. No OOM kill, 0 restarts. `FAULT_INJECT=1` returned `HTTP 500 {"error":"invalid_model_output"}`$^{\mathbf{M}}$. *Problem found:* the first version took 9.8 s per request at 1 CPU$^{\mathbf{M}}$. Profiling showed the oneDNN (mkldnn) kernels in the arm64 Linux torch wheel are pathologically slow for batch-1 decoding; disabling them (`torch.backends.mkldnn.enabled = False`) brought it to 0.43 s$^{\mathbf{M}}$, a 23× speed-up.

4 Task 4 — Kubernetes deployment, scaling and reliability

`k8s/app/parser.yaml`: namespace `app`; Deployment with 2 replicas, requests 500m CPU / 512 Mi, limits 1 CPU / 1 Gi; liveness probe on `/healthz` (every 10 s, 3 failures) and readiness probe on `/readyz` (every 2 s, 1 failure); rolling update `maxUnavailable: 0, maxSurge: 1`; a `preStop` sleep of 8 s so a pod keeps serving requests already routed to it while it is removed from the endpoints; ClusterIP Service; Traefik Ingress routing `/parse`; HPA (`autoscaling/v2`) min 1, max 2, 70% average CPU, scale-down stabilisation 60 s. Experiments are scripted in `scripts/k8s_tests.sh`; outputs in `results/task4/`. (The HPA, rolling-update, recovery and load-balancing runs were done before I switched the stored weights to float16; they served the float32 copy of the same v1/v2 models, which gives identical outputs. Cold start and Task 7 were re-run on the float16 copy.)

Test	Result$^{\mathbf{M}}$
Ingress	`curl -X POST localhost:8080/parse` returns the frame with both headers (`resources.txt`).
No endpoints while loading	Scaled 0→1 and polled each second: pod `/readyz`=503, EndpointSlice `ready=false`, Ingress `/parse` = HTTP 503, until the log line "model v1 ready", then endpoint `true` and 200 (`probes.txt`).
HPA	Idle at 2 replicas, 0% CPU ⇒ scaled to 1 (05:06:40). Under the 4-user load the single pod hit 990m (198% of its request) ⇒ *New size: 2* at 05:07:21. After the load ended CPU fell to 0% and the HPA scaled back to 1 (event "All metrics below target", 05:11:49) (`hpa.txt`).
Rolling update	During a 5-min load test: `kubectl set env` to `MODEL_VERSION=v2`, `MODEL_URI=s3://models/v2/`, then `kubectl rollout undo`. **1,314 requests, 0 failures (0.00%)**; v2 served 307 of them; after the rollback 5/5 probes returned `X-Model-Version: v1` (`rollout.txt`).
Recovery	Deleted one of two pods under load: Ready count went 2→1→2 in 7 s, the old pod was gone after 11 s; **774 requests, 0 failures** (`recovery.txt`).
Cold start	Deleted all pods 3 times. Pod creation→Ready (slowest pod per run): 4, 4, 4 s, **median 4 s** (1 s timestamp resolution). Logs show `downloading model s3://models/v1/ ...download finished: 144.4 MB in 0.2 s ...model v1 ready` (`coldstart.txt`).
Load balancing	Exactly two Ready replicas (HPA min pinned to 2 for the run), standard command through the Ingress: **253 (49.8%) and 255 (50.2%)** of 508 successful requests (`balance.txt`).

Limitations of the local cluster. Both k3d "nodes" are containers on one laptop: losing the laptop, the Docker VM (which actually froze once) or the disk loses everything, so this shows *pod*-level recovery, not fault tolerance across machines or zones. SeaweedFS is a single replica on a local-path volume: a single point of failure. metrics-server samples every 15 s and the HPA syncs every 15 s, so scale-up takes about 40–60 s; `max 2` replicas is the ceiling. CPU "utilisation" is relative to the 500m request, so a pod at its 1-CPU limit shows 200%. Pods are throttled by the shared host CPU, so other load on the laptop changes the results.

5 Task 5 — Subnetting and network access control

Largest first from 10.20.0.0, smallest prefix with $2^{32-p} - 2 \geq$ hosts (working in `network/subnets.md`):

Subnet	Needed	Prefix	Network	Usable range / Broadcast	Usable
app	1,000	/22	10.20.0.0	10.20.0.1–10.20.3.254 / 10.20.3.255	1,022
public	250	/24	10.20.4.0	10.20.4.1–10.20.4.254 / 10.20.4.255	254
data	100	/25	10.20.5.0	10.20.5.1–10.20.5.126 / 10.20.5.127	126
management	20	/27	10.20.5.128	10.20.5.129–10.20.5.158 / 10.20.5.159	30

NACL for the public subnet (`network/nacl.json`). Inbound: 90 deny *all* from 198.51.100.0/24; 100 allow TCP 80 and 110 allow TCP 443 from 0.0.0.0/0; 120 allow TCP 22 from 10.20.5.128/27; 130 allow TCP 1024–65535 from 10.20.0.0/22 (replies from app to the load balancer's ephemeral ports); everything else falls to the default deny. Outbound: 100 allow TCP 8000 to 10.20.0.0/22; 110 allow TCP 1024–65535 to 0.0.0.0/0 (replies to clients). `nacl_sim.py -check`: **6/6 correct**$^{\mathbf{M}}$.

Why replies need rules, and why order matters. A NACL is *stateless*: it keeps no record of connections, so a reply packet is checked against the rules like any new packet. The reply to a client's request to port 443 leaves for the client's ephemeral port (1024–65535) and needs outbound rule 110; the app's reply to the load balancer's call to port 8000 returns to the load balancer's ephemeral port and needs inbound rule 130. (A stateful security group would allow these automatically.) Rules are evaluated from the lowest number and the *first match wins*. A packet from 198.51.100.9 to port 443 matches both "deny 198.51.100.0/24" and "allow 443 from anywhere"; it is blocked only because the deny is rule 90, lower than the allow rules 100/110. Numbered, say, 200, the abusive range would get in on the web ports.

6 Task 6 — Reproducibility, testing and CI/CD

Fresh clone in three commands: `git clone ...`, `cd cc-kd-command-parser`, `make up`. `make up` creates the cluster from `cluster/k3d.yaml`, generates credentials into Secrets, deploys SeaweedFS, builds both images and imports them into k3d, runs a seed Job that creates the buckets, uploads the pool and copies the v1/v2 Release tarballs (after checking their SHA-256) into `s3://models/`, and deploys the app. `make down` deletes the cluster. I tested this after `make down`, from a fresh clone into an empty directory: `make up` exited 0 and `/parse` answered through the Ingress with `X-Model-Version: v1` (`results/task6/make_up_fresh_clone.log`)$^{\mathbf{M}}$. It took 22.5 min$^{\mathbf{M}}$, almost all of it downloading the k3s system images and the two 127 MB Release tarballs over my ≈0.5–1 MB/s link. A first attempt timed out waiting for SeaweedFS while k3s was still pulling its system images, so I raised the timeouts. Prerequisites: Docker, k3d, kubectl, make, openssl (and Python 3 for the training/test targets).

GitHub Actions (`.github/workflows/ci.yml`), stages in order: *lint* (ruff) → *unit-tests* (pytest, 13 tests: sharding, the cleaning rules and determinism, reproducible packaging, service endpoints/headers/validation/fault injection) → *evaluate* (download the Release with `gh release download`, verify the checksum, run the student on `validation.jsonl`, `score.py`) → *quality-gate* (`score.py -min-validity 95`); plus a parallel *secret-scan* job (gitleaks over the full history). A manual input `model=untrained` evaluates the base Pythia-70M instead. **Runs:** passing run on the student v1 Release (checksum `OK`, validity 100, full frame 45; same numbers on GitHub's x86 runner as on my laptop): https://github.com/Manas-Trivedi/cc-kd-command-parser/actions/runs/42317803924. **Gate doing its job** (untrained Pythia-70M, `QUALITY GATE FAILED: json_validity 0.0 < 95.0`; lint, tests and evaluation passed, the quality-gate job failed): https://github.com/Manas-Trivedi/cc-kd-command-parser/actions/runs/42318098565. gitleaks scans the full git history and reports no leaks (CI job and `results/task6/gitleaks_local.txt`); the only secrets on disk are in the git-ignored `.secrets/`. Evidence: `results/task6/`.

7 Task 7 — Performance, cost and review

Standard command (4 users, 2 min, through the Ingress) against the deployment in its normal state (HPA at 1 replica when idle). Files: `results/task7/`.

Metric	Value[M]	Source
Requests / failures	467 / 0 (**error rate 0.00%**)	`loadtest_stats.csv`
Latency p50 / p95	**1,300 ms / 1,700 ms** (min 405, max 1,880)	`loadtest_stats.csv`
Throughput	**3.91 req/s** (4.3–4.4 req/s once 2 replicas were up)	`loadtest_stats.csv`
Pod CPU / memory	pod 1: 980–991m, 583–620 Mi; pod 2 (added by the HPA during the run): 801–998m, 588–613 Mi	`perf_top.txt`
Cold start	median 4 s creation→Ready (Task 4)	`task4/coldstart.txt`

Both pods run at their 1-CPU limit: the service is CPU-bound. One request takes ≈0.4 s of CPU time (≈25 greedy decode steps), so 4 concurrent users queue behind the inference lock, which explains a 1.3 s median. The replica split was 267/202 because the second pod only started partway through the run.

Cost (rate card[A]: 3.50 INR per vCPU-hour, 0.45 INR per GiB-hour; billed on limits). Per replica: 1 vCPU × 3.50 + 1 GiB × 0.45 = 3.95 INR/h. Under sustained load the HPA runs 2 replicas: 2 × 3.95 = 7.90 INR/h. Requests per hour at the measured 3.9116 req/s: 3.9116 × 3600 = 14,082. Cost per 1,000 requests = 7.90/14,082 × 1000 = **0.561 INR**. (At the 2-replica steady-state throughput of 4.39 req/s[M]: 7.90/15,804 × 1000 = 0.50 INR.) The cost is dominated by the CPU limit (89% of the hourly price).

SLO. *99% of `/parse` requests return HTTP 200 within 30 s, measured over one standard load-test run.* Window: one 2-minute run (467 requests). Error budget = 1% × 467 = 4.67, i.e. **4 failed requests** (rounded down). Used: 0 failures = 0% of the budget[M]. (Rolling update and recovery runs also used 0 of their budgets of 13 and 7 requests.)

Review.

Concern	Evidence (measured)	Limitation found	Improvement proposed
Data and storage	2,772 of 3,000 replies kept; 162 rejected for invented slots; labelling took 76 min with 3 workers	Single-replica SeaweedFS on a local volume; wrong-but-valid teacher labels pass the cleaning	Replicated object store (3 volume servers); a second teacher pass or self-consistency vote to flag suspicious labels
Model quality	full frame 45% (teacher 54%), JSON validity 100% on validation	Weak on places (82%) and intent (86%); copies teacher errors	Train on all 2,772 records; constrained decoding of the intent enum; better teacher prompt
Latency and throughput	p50 1,300 ms, p95 1,700 ms, 3.9 req/s at 4 users	One request at a time per pod, CPU-bound greedy decoding	Batch concurrent requests, int8 quantisation or ONNX Runtime, KV-cache reuse of the fixed prompt prefix
Resource utilisation	pods at 980–998m of 1 CPU, 583–620 Mi of 1 Gi	CPU saturates at 4 users; float16→float32 conversion raises load-time memory to ≈790 MiB	Store float32 (or load with `low_cpu_mem_usage`) to cut the peak; raise HPA max and lower the target so it scales before saturation
Reliability and recovery	0 failures in rolling update (1,314 req) and pod deletion (774 req); cold start 4 s	One laptop, one storage pod: no machine-level fault tolerance	Multi-node cluster across hosts, PodDisruptionBudget, anti-affinity, replicated storage
Security and cost	gitleaks: 0 findings; 0.561 INR per 1,000 requests	Plain-HTTP ingress, one S3 admin key for all workloads, billing on limits while idle	TLS on the Ingress, per-workload read-only S3 keys, NetworkPolicies, scale to 1 replica when idle (already via HPA)

References

Kim & Rush, *Sequence-Level Knowledge Distillation*, EMNLP 2016. Hinton, Vinyals & Dean, *Distilling the Knowledge in a Neural Network*, 2015. Bastianelli et al., *SLURP*, EMNLP 2020 (CC BY 4.0). Biderman et al., *Pythia*, 2023 / EleutherAI Pythia-70M model card. Documentation: Hugging Face Transformers & TRL, k3d, K3s, SeaweedFS, Kubernetes (probes, HPA, Deployments, Indexed Jobs), AWS VPC Network ACLs, Google SRE book (SLOs).